Rev.03
Redatto da Marco Pellino, Menerva s.r.l.s.

Manuale del linguaggio Manager e del framework ADAS



Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

menerva

Sommario

Manuale del linguaggio Manager e del framework ADAS 1
Premessa 3
Il linguaggio Manager 4
Token 6
Tipi base 9
Sintassi in formato BNF del linguaggio Manager 10
Categorie sintattiche e funzioni 12
Istruzioni di selezione 13
Istruzioni iterative 14
Le espressioni 14
Tipi nullable 14
Il servizio di esecuzione. 15
Le primitive 16
Il linguaggio 18
Le variabili 19
Ulteriori esempi di codice 20
Framework ADAS 22
Editor visuale di codice 26
Riferimenti 37

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

menerva

Premessa

A cura di Enrico Eugeni.

L'obiettivo che ci ha spinto allo sviluppo di un linguaggio proprietario è quello di distribuire un linguaggio semplice e consentire la configurazione dell'acquisizione di dati ambientali mediante un sistema integrato.

Avendo definito lessico con pochi tokens ed una sintassi semplice, qualunque utente è in grado di poter sviluppare codice senza essere necessariamente uno sviluppatore o avere una base di informatica.

Il linguaggio manager è un linguaggio di dominio e come tale ha specifici costrutti per la gestione di risorse e la descrizione dei contesti di utilizzo.

Inoltre, il codice agile e scalabile permette di adattarsi al variare delle normative che riguardano la gestione di dati ambientali.

In questo modo siamo arrivati allo sviluppo del linguaggio Manager e del suo ambiente di gestione ADAS.

ADAS (Advanced Data Analytic System) è il framework in cui sono eseguiti i programmi Manager e permette la gestione delle configurazioni, esecuzione e gestione dei dati acquisiti mediante i programmi Manager.

In questo manuale è descritto il lessico, la sintassi e la semantica del linguaggio Manager e le funzionalità del framework ADAS.

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

Il linguaggio Manager

L'analisi lessicale e sintattica del linguaggio Manager è stata generata mediante GNU Bison [1]. Bison è un generatore di parser, legge in un formato simile a YACC una grammatica libera dal contesto e genera un parser scritto in codice C, C++ o Java. Di default genera parser del tipo LARL, ma può anche essere configurato per generare parser LR, IERL e GLR. [2]. Nel caso specifico del linguaggio Manager il codice viene generato in C++. Il codice scritto in Manager ha diverse categorie sintattiche comuni a molti linguaggio di programmazione come: direttive, dichiarazioni di variabili tipate, funzioni o procedure, comandi e operazioni di I/O. Il linguaggio Manager appartiene alla classe dei linguaggi imperativi ed è case sensitive.

Il linguaggio Manager permette la definizione di elementi concorrenti indicati come Job. Un Job permette l'acquisizione dei valori ambientali.

Segue uno schema del tempo di esecuzione di un programma.

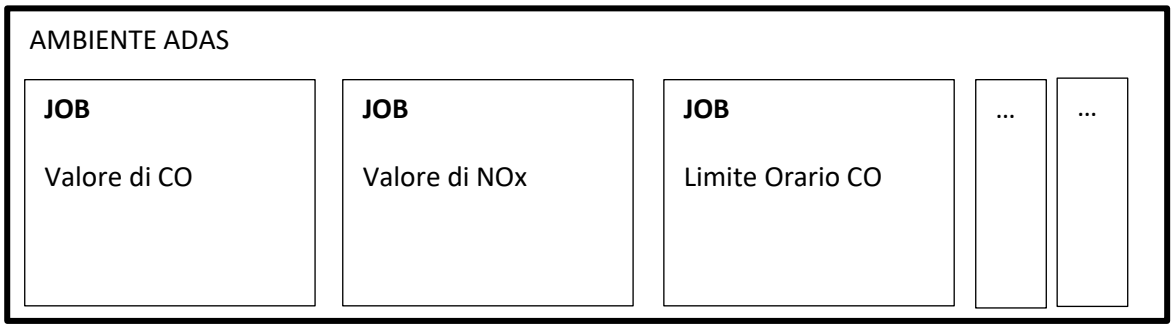


Fig.1 Job nell'ambiente ADAS

Ogni Job viene eseguito indefinite volte dall'ambiente di esecuzione ADAS. All'interno di ogni Job una proprietà ne determina il tempo di esecuzione. La variabile di riferimento del Job è pubblica e può essere letta e modificata da ogni altro Job presente nell'ambiente ADAS previa dichiarazione e da un modulo Viewer presente in ADAS.

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

I vari Job sono assegnati ai Device che realizzano le acquisizioni.

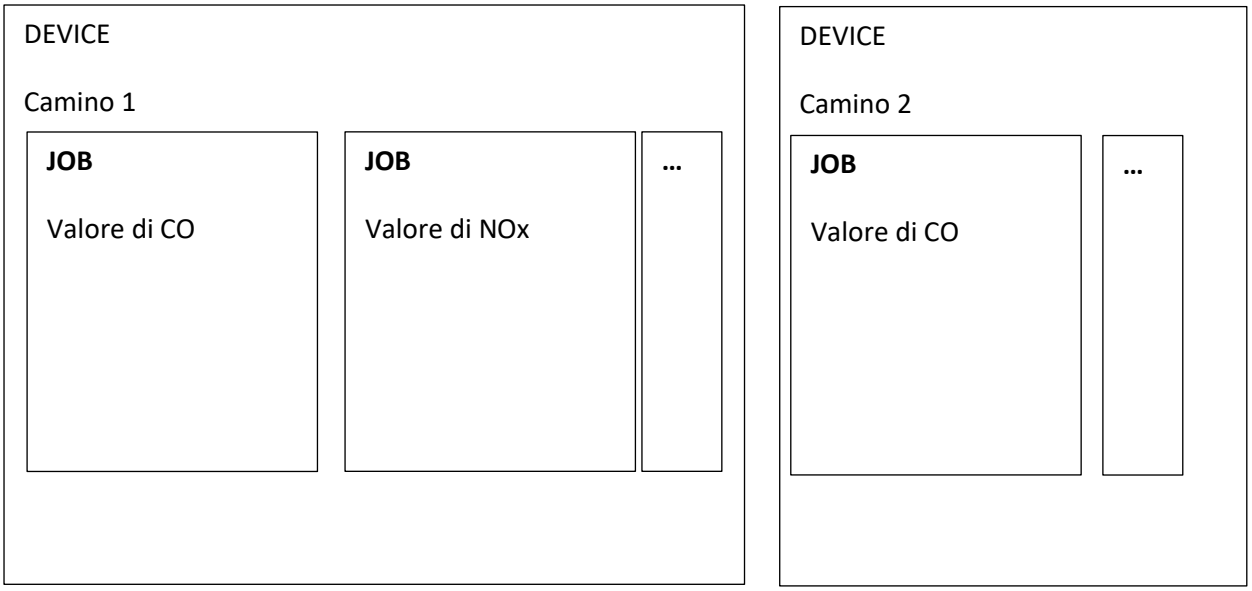


Fig.2 Dichiarazione di Job nell'ambiente ADAS.

I nomi dei Job all'interno delle Device devono essere univoci.

Ogni Device ha definito le tempistiche di acquisizione mediante il modulo Viewer. Sempre con il Viewer viene definito il ciclo di elaborazione del Job generalmente superiore al primo.

Da questa rapida panoramica, prima di entrare nel dettaglio si evince che il linguaggio Manager non è un linguaggio orientato ad oggetti ma ha predefiniti tipi specifici di dominio assimilabili a classi: Job e Device ai quali sono associate specifiche funzionalità e operazioni (attributi e metodi).

Il linguaggio inoltre permette l'uso di primitive predefinite.

Una variabile viene dichiarata, definita e possibilmente inizializzata nello stesso costrutto. Non è necessario definirne il tipo poiché viene dedotto dalla sua inizializzazione. Il linguaggio ha tipizzazione forte e statica.

Le variabili definite in altri Job devono essere dichiarate prima essere usate.

Tipi predefiniti in Manager sono interi e reali.

I vari Job hanno una sequenza di esecuzione definita da ADAS.

Ogni Job è racchiuso dalle parole chiavi **progbegin** e **progend**. Un Device può contenere diversi Job.

In un programma Manager non possono essere dichiarati Device con lo stesso nome.

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

menerva

Token

I token rappresentano gli elementi base di ogni linguaggio. Ciascuna riga della tabella raggruppa token per tipologia di uso.

TABELLA 1 –TOKEN DEL LINGUAGGIO MANAGER

DESCRIZIONE	TOKEN
Spazio bianco	Spazio
Tabulazione	Horizontal_tab
Punteggiatura	. ;
Terminatore riga	Ritorno carrello, carattere riga successiva, ;
Commenti	#
Blocco	Else, end if ( )
Operatore matematico	* / ^ + - > <
Operatore composito	:= == != #= %= = @
Operatori binari	EQ NEQ AND OR

Spazio bianco

Lo spazio bianco è definito come carattere spazio. Separa le istruzioni nel codice. È essenziale l'utilizzo della spaziatura per la corretta compilazione del codice. Ad esempio:

```
"CO"="CO"-65535;
```

Genererà un errore di compilazione. Lo spazio dev'essere utilizzato per separare le istruzioni in una riga di codice.

Tabulazione

I caratteri di tabulazione servono per la spaziatura del codice e per indentare il codice. Non sono essenziali per la compilazione del costrutto.

Punteggiatura

La punteggiatura separa e delimita i costrutti del linguaggio Manager. Nello specifico il punto e virgola (;) è essenziale in quanto carattere terminale di una riga di codice, dev'essere utilizzato in chiusura delle istruzioni. Fanno eccezioni le istruzioni di selezione e le istruzioni iterative.

```
if("CO" > ("CO_SI_E01" + "toll_sup_co_E01")) then
  "CO".flag = 21;
  "CO" = 125.75;
  setquality(192);
  "CO" = "CO_SI_E01" + "toll_sup_co_E01";
else
   noop;
fi;
```

Il punto (.) viene utilizzato come separatore decimale nei valori numerici e come carattere di accesso alle proprietà delle primitive, come si può vedere nell'esempio precedente il punto è utilizzato sia per assegnare un valore numerico decimale che per accedere alla proprietà flag del valore del job CO.

Terminatori di riga

I terminatori di riga sono l'a capo (INVIO, carattere di escape, ritorno di riga) e il punto e virgola (;).

L'a capo dev'essere utilizzato nelle istruzioni iterative e di selezione.

```
if("CO" > ("CO_SI_E01" + "toll_sup_co_E01")) then(ACAPO)
(TAB)"CO".flag = 21;
```

Il punto e virgola chiude le istruzioni.

Commenti

Il linguaggio Manager supporta i commenti. Il token per i commenti è il carattere cancelletto (#). Il cancelletto dev'essere utilizzato come prima istruzione della riga, non può essere preceduto da nessun carattere, nemmeno uno spazio.

```
#SOGLIA INFERIORE
#-------------------------------------
#Controllo soglia inferiore se CO compreso nella tolleranza lo riporto
inizio
#scala
if(("CO" < "CO_ZI_E01") && ("CO" > ("CO_ZI_E01" - "toll_inf_co_E01")))
then
    "CO" = "CO_ZI_E01";
else
     noop;
fi;
```

Racchiudere un blocco di codice

Soltanto le istruzioni iterative e di confronto possono racchiudere dei blocchi di codice. Per farlo si utilizza la sintassi mostrata ai capitoli relativi a tali istruzioni. Di coseguenza un blocco IF (espressione) then racchiude tutte le istruzioni al suo interno fino all'else, poi le istruzioni dall'else all'fi.

Lo stesso abbiamo nel blocco While Do, le istruzioni sono racchiuse dal blocco WHILE e vengono eseguite fino al token DO.

Anche le parentesi possono essere utilizzate per racchiudere un blocco di codice, vanno utilizzate all'interno di una singola linea di codice e permettono di determinare la priorità in cui saranno risolte le operazioni al loro interno.

Operatore matematico

Gli operatori matematici sono moltiplicazione (*), divisione (/), addizione (+), sottrazione (-) ed elevamento a potenza (^).

L'ordine con cui vengono risolti gli operatori matematici è il seguente:

TABELLA 2 – ORDINE OPERATORI

ORDINE	OPERATORE
1	AND OR
2	> <
3	- +
4	* /
5	^

Altri operatori matematici sono:

sin seno
cos coseno
atan arcotan
acos arcocoseno
asin arcoseno
ln logaritmo
exp elevamento a potenza
sqrt radice quadrata

Operatore composito

Gli operatori compositi sono utilizzati nelle assegnazioni di valori e nelle istruzioni iterative e comparative.

Nel dettaglio:

TABELLA 3 – DESCRIZIONE OPERATORI COMPOSITI

OPERATORE	DESCRIZIONE
:=	Assegnazione di tempo, valore e flag
#=	Assegnazione di tempo
=	Assegnazione di valore
%=	Assegnazione di opc quality
==	Comparazione di uguaglianza
!=	Comparazione di disuguaglianza
@	Operatore per referenziare un device e accedere ai job all'interno di tale device

Operatori binari

Gli operatori binari sono utilizzati per le comparazioni bitwise e per le condizioni all'interno delle istruzioni comparative.

Nel dettaglio:

TABELLA 4 – DESCRIZIONE OPERATORI BINARI

OPERATORE	DESCRIZIONE
||	Or logico, basta il verificarsi di una condizione per restituire true
&&	And logico
!=	Non uguaglianza
==	Uguaglianza

Tipi base

Binary.

Alla base di tutto vi è il tipo binary. Gli oggetti di tipo binary possono assumere valori zero e uno.

Analogic

Il tipo analogic è un tipo SINGED DOUBLEWORD e può assumere valori da -2^63 a +2^63.

Device

Un device è un contenitore di job con il proprio tempo di esecuzione. Svolge anche ulteriori funzioni ed implementa altre funzioni che vengono esposte nel capitolo Primitive.

Job

Un Job contiene un valore composto di VALORE che può essere di tipo Binary o Analogic come esposto in questo capitolo, il timestamp di lettura, la qualità OPC e il flag di validità.

Ha un proprio tempo di esecuzione che viene utilizzato successivamente a quello del tempo del Device a cui appartiene.

Per cui un device con tempo 5 secondi esegue i suoi ogni 5 secondi, ma se al suo interno un job ha tempo 60 secondi, ogni 60 secondi sarà eseguito il codice all'interno dell'istruzione elapsed(). Questo accenno è necessari per comprendere quali sono i tipi base del linguaggio. In merito al funzionamento delle proprietà e delle funzioni implementate nei tipi base consultare il capitolo successivo.

Sintassi in formato BNF del linguaggio Manager

La sintassi del linguaggio Manager è specificata qui di seguito in EBNF [3]

```
<program> = 'progbegin' {<directive>}* {<declaration>}* {<command>}* 'progend'

<directive> = 'declare' <description> ';'

<declaration> =
      | <identifier>
      | <identifier> '=' <numer>
      | 'declare' <description>
      | 'declare' <description> '=' <number>

<command> = 'noop' ';'
      | 'print' <description> ';'
      | 'return' <exp> ';'
      | <descrition> '=' [<exp> | <description>] ';'
      | <descrition> [':=' | '#=' | '%='] <description> ';'
      | <descrition> '.' 'time' '=' <description> '.' 'time' ';'
      | <descrition> <'.'> 'flag' '=' [<description> '.' <flag> | <exp> ]';'
      | <procedure0> '()' ';'
      | <procedure1> '(' <exp> ')'n ';'
      | <procedure2> '(' <exp> ',' <exp> ')' ';'
      | 'if' '(' <bexp> ')' 'then' <commands> 'else' <commands> 'fi' ';'
      | 'while' <bexp> 'do' <commands> 'done'

<exp> = [<bexp> | <aexp>] |  '(' <exp> ')'

<aexp> = <number>
      | <desxcription>
      | <desxcription> '.' 'flag'
      | <function0> '()'
      | <function1> '(' <exp> ')
      | <function2> '(' <exp> ',' <exp> ')'
      | <function4> '(' <exp> ',' <exp>',' <exp> ',' <exp> ')'
      | '-' <exp>
      | <exp> ['*' | '/' | '^'] <exp>

<bexp> =    '1' | '0' | <exp> ['&&' | '||' | '=='|'!=' | '>' | '<' ] <exp>

<procedure0> = 'setnan' | 'ClearForced'

<procedure1> = 'doRangeCheck' | 'setforced' | 'setquality' | 'store' | 'storep' |
'setdevstatus' | 'setjobstatus' | 'setmonitor'

<procedure2> = 'setalert' | 'setwarning' | 'bitset'

<function0> = 'elapsed' | 'isforced' | 'forcedval' | 'recovering' | 'getsecond' |
'getsec' | 'getminute' | 'gethour' | 'getday' | 'getweekday' | 'getmonth' |
'getyear' | 'getproto' | '

<function1> = 'isfinite' | 'sin' | 'cos' | 'atan' | 'acos' | 'asin' | 'ln' | 'exp' |
'sqrt' | 'int'

<function2> = 'bittest' | 'mmean'
```

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

```
<function4> = 'StartTask' | 'bintoreal' | 'ping'
```

Token speciali descritti con espressioni regolari[4]

```
<identifier> = [a-zA-Z][a-zA-Z0-9_]*

<description> = "[a-zA-Z0-9_\s]+"

<number> -?[0-9]*(\\.[0-9]+)?
```

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

menerva

Categorie sintattiche e funzioni

Segue la lista dei costrutti principali:

TABELLA 5 – DESCRIZIONE COSTRUTTI

OPERATORE	DESCRIZIONE
declare	utilizzato per istanziare una variabile.
depend	utilizzato per richiamare il codice di un'altra classe-funzione
do	utilizzato in chiusura di un ciclo while
while	inizio di un ciclo while
if	inizio di una condizione if
then	chiude la condizione da verificare all'interno della dichiarazione di un if
else	controparte negativa della condizione if
fi	chiude la condizione if
noop	utilizzato all'interno della condizione if indica che al seguente punto non è da eseguire alcuna operazione
print	trasmette un informazione
progbegin	inizio di un set di istruzioni
progend	fine di un set di istruzioni
return	ritorna una variabile

Di seguito invece le funzioni della classe job nel dettaglio:

TABELLA 6 – DESCRIZIONE COSTRUTTI

OPERATORE	DESCRIZIONE
print	traccia nel log il valore di una Job es. print CO;
flag	assegna il flag es. CO.flag = 10; es. CO.flag = "CO raw".flag;
prev	contiene il valore precedente di una Job es. if (CO == CO.prev) then
isfinite	indica se una Job contiene un valore finito (not NaN)
elapsed	indica se è trascorso il periodo di campionamento configurato nelle proprietà della primitiva
isforced	Operazione I/O, indica se il valore del job è impostato dall'esterno.
forcedval	Operazione I/O, preleva il valore impostato dall'esterno.
setforced	Operazione I/O, setta il valore impostato dall'esterno.
setquality	setta l'opcqaulity della Job
store	salva nel db con il tempo reale
storep	salva nel db con il tempo reale ma arrotondato
setnan	setta a NaN
setalert	Operazione I/O, invia segnalazione di alert
setwarning	Operazione I/O, invia segnalazione di warning
bittest	effettua il test di un bit di es. if bittest("ingressi digitali",0) == 1 then fa il test del primo bit del valore della Job
bitset	setta il bit di una word

getsecond	secondi passati dalla mezzanotte (0 - 86400)
getsec	secondo attuale (0 - 59)
getminute	minuto attuale (0 - 59)
gethour	ore passate dalla mezzanotte (0 - 23)
getday	giorno del mese (1 - 31)
getweekday	giorno della settimana: 0 domenica, 1 lunedi..
getmonth	mese (1 - 12)
getyear	anno
getproto	indice del protocollo in uso
ping	es. if ping(192,168,1,2) then. Esegue un ping verso un indirizzo IP

Istruzioni di selezione

Un istruzione di selezione valuta un'espressione per determinare quale ramo di codice viene eseguito successivamente. Le istruzioni di selezione comprendono le istruzioni if e i cicli while.

Un'istruzione if viene valutata a un'espressione booleana. Se l'espressione è true, il controllo viene trasferito alla successiva istruzione true. Se l'espressione è false il controllo viene trasferito alla sezione else dell'if.

Ecco la sintassi di un istruzione if:

```
If (bittest("ALLARMI", 1) == 1) then
```

Di conseguenza, se la condizione si verifica verrà eseguita la sezione dopo il then, viceversa la sezione dopo l'else.

```
If (bittest("ALLARMI", 1) == 1) then
"CO".flag = 10;
else
"CO".flag = 0;
fi;
```

Le istruzioni if possono essere nidificate come segue:

```
If (bittest("ALLARMI", 1) == 1) then
"CO".flag = 10;
else
      If (bittest("ALLARMI", 2) == 1) then
      "CO".flag = 40;
      else
      "CO".flag = 0;
      fi;
fi;
```

Il costrutto noop che potremmo tradurre come no operation indica che in quella porzione di codice non c'è da eseguire nessun codice.

Istruzioni iterative

IL linguaggio Manager dispone di un'unica espressione iterativa. L'istruzione `while` viene eseguita da zero a più volte e viene eseguita quando l'espressione booleana che la compone rimane vera. Se l'espressione booleana è falsa l'iterazione del while non verrà affatto eseguita.

La sintassi è la seguente:

```
WHILE (espressione booleana)
codice
DO;
```

L'istruzione DO chiude il ciclo while e ne ripete il ciclo fino a che l'espressione booleana non diventa falsa.

Le espressioni

Le espressioni vengono risolte ad un valore. Un'espressione comunemente contiene uno o più operatori. Tuttavia un'espressione può anche essere un singolo valore o una singola costante. Gli operatori sono unari, binari o ternari.

Ad eccezione dell'assegnazione e degli operatori ternari, le espressioni vengono valutate da sinistra a destra. Un'espressione può contenere più operatori, gli operatori vengono valutati in ordine di precedenza, per modificare la precedenza si utilizzano le parentesi tonde.

TABELLA 7 – ORDINE DI PRECEDENZA DEGLI OPERATORI

PRECEDENZA	**Operatore**
1	Funzione, incremento, dichiarazione di variabile
2	Operazioni addizione unaria +, -
3	Operatori divisione, modulo e moltiplicazione
4	Operatori di shift
5	Operatori di comparazione, == o non uguale !=
6	Operatori logici AND e OR
7	Assegnazioni

Tipi nullable

Assegnare null ad un offetto indica che è inutilizzato. Ciò è consistente per tutti i tipi reference. Si può segnalare analogamente un intero come inutilizzato? Come si può stabilire che un tipo value non contiene nulla? Non è possibile assegnare direttamente un tipo null.

```
"CO".value = NaN;
```

Questa istruzione restituirebbe un errore di compilazione.

Vediamo quindi come assegnare un valore null.

Una prima opzione potrebbe essere quella di assegnare -1 ad un valore di tipo Analogic oppure un valore 0 ad un valore di tipo binary. Ma sarebbe corretto?

In un caso studio una Job acquisita, poniamo un valore di un inquinante come il CO potrebbe avere un valore inferiore allo zero nel caso lo strumento che acquisisce tale parametro vada in rottura cavo. Oppure, un digitale, poniamo ad esempio uno stato di un sensore come Alta temperatura linea, potrebbe essere disattivo, quindi avere valore 0.

Per ovviare a questo problema la primitiva job implementa la funzione setnan. La funzione setnan imposta il valore a nullo della primitiva. In questo modo è possibile assegnare un null ad una primitiva senza incorrere in un eccezione o un errore di compilazione.

Eccone un esempio:

```
If (bittest("ALLARMI", 1) == 1) then
setnan();
else
noop;
fi;
```

Questo snippet di codice controlla lo stato di un digitale, quello dell'impaccato ALLARMI e nel caso il valore del primo bit del digitale impaccato ALLARMI risulti ad 1 imposta a NULL il valore della primitiva job in questione.

TABELLA 8 – DIAGRAMMA DI STATO DEL VALORE DEL CO

JOB	**Valore**
CO	150
ALLARMI	2^1
Conseguentemente	
CO	NaN

Dalla tabella 8 possiamo vedere come al verificarsi della condizione il valore di CO diventa NaN.
Abbiamo espresso il valore della primitiva job ALLARMI come 2^1, questo perché un digitale impaccato è un tipo particolare di job in cui il valore corrisponde ad un range da 2^0 fino a 2^15. Un impaccato è di conseguenza un job di tipo digitale impaccato che può essere testato bit a bit. La funzione bittest svolge questa funzione, testa un valore binario in una WORD (16bit). La funzione bittest è un operatore AND bitwise. Ma approfondiremo le funzioni delle primitive successivamente.

Il servizio di esecuzione.

Una nota relativa al servizio di esecuzione è essenziale per comprendere l'IDE di sviluppo e il tempo di esecuzione del codice scritto in Linguaggio Manager.

Il servizio che esegue il codice Manager è un servizio che può essere eseguito in ambiente Windows e Linux e si occupa di eseguire le istruzioni scritte nel linguaggio nativo. Ogni Job viene eseguito sulla base del suo tempo di esecuzione. I Job si dividono in più tipologie. Abbiamo i Job di tipo analogico, digitale, uscita analogica, uscita digitale ed elaborazione. Ogni set di Job fa parte di una classe che le racchiude. Tale classe si chiama Device.

Le primitive

Esaminiamo nel dettaglio le primitive che compongono il linguaggio.

Device.

La primitiva Device è una classe. È composta da un vettore contenente i Job, il tempo di esecuzione dell'oggetto, interpretato dal servizio Manager e da altre variabili.

Segue la lista delle variabili che compongono la classe ordinate per visibilità.

Variabili pubbliche.

TABELLA 9 – MEMBRI PUBBLICI DELLA PRIMITIVA DEVICE

MEMBRO	DESCRIZIONE
m_workSet	Area di lavoro del Device
m_driver	Driver del Device
m_database	Database di appartenenza del device
m_recordId	Record ID del device
m_jobs	Array dei job
m_name	Nome
m_ptPosition	Puntatore del device
m_LineName	Nome della linea del device
m_Address	Indirizzo del device
m_DriverName	Nome del driver
m_ProtoIndex	Indirizzo del protocollo
m_StatusJob	Status di acquisizione dei job
m_AcqEndJob	Termine delle acquisizioni dei job
m_IgnoreErrors	Booleano, indica se ignorare gli errori da mostrare a video
m_thread	Thread di esecuzione del device
m_CriticalSection	
m_TaskEvent	Evento in corso
m_StopEvent	Ferma un evento
m_StopDlg	Comando di stop
m_CancelRqst	Richiesta di stop
m_faults	Errori
m_alarmCount	Contatore di allarmi
m_alarmBlink	Indica se un allarme deve lampeggiare
m_EventTime	Tempo di un evento occorso
m_EventCode	Codice dell'evento occorso
m_EventDescription	Descrizione dell'evento occorso
m_EventOrigin	Origine dell'evento occorso
m_DriverPrivate	Caratteristiche del driver
m_config	Variabile interna per gestione della configurazione del device
m_timeConfiguration	Tempo della configurazione
m_timeLastDownLoad	Ultima volta che la configurazione è stata salvata
m_timeNextDownLoad	Prossima esecuzione del salvataggio della configurazione

m_timeNextSnapshot	Prossima snapshot dei job in lettura
m_DownLoadType	Tipo di download della configurazione
m_DownloadIndex	Indice di download della configurazione
m_AutoDownLoadInterval	Tempo per il download automatico della configurazione
m_overwrite	Flag per sovrascrittura
m_ProgramArray	Vettore contenente le istruzioni del device
m_SnapshotPerioD	Tempo di aggiornamento dell'acquisizione. Corrisponde al tempo di refresh della Device
m_LastAccessTime	Ultimo accesso alla configurazione
m_ConnectDuration	Durata della connessione
m_VirtualTime	Tempo virtuale del device
m_Factors	Fattori di correzione

Variabili protette.

TABELLA 10 – MEMBRI PROTETTI DELLA PRIMITIVA DEVICE

MEMBRO	DESCRIZIONE
m_modified	Indica se il codice di un device è stato modificato o se è stato modificato il codice di un job interno al device
m_erased	Indica se è stato cancellato il device
m_JobAlarm	Status degli allarmi (output I/O) del device

Job.

La primitive Job è una classe. Al suo interno implementa codice scritto in linguaggio Manager e può assumere valori di tipo binary o analogic. La primitiva Job può essere tipizzata e può assumere valori nullable. Il valore della primitiva non è legato soltanto ad un valore numerico ma è composto anche dal tempo come timestamp e flag.

Segue la lista delle variabili che compongono la classe ordinate per visibilità.

Variabili pubbliche.

TABELLA 11 – MEMBRI PUBBLICI DELLA PRIMITIVA JOB

MEMBRO	DESCRIZIONE
m_device	Device a cui il job appartiene
m_type	Tipo di job
m_subType	Sottotipo di job
m_name	Nome del job
m_period	Tempo di esecuzione del codice
m_period_idx	Indice del tempo di esecuzione
m_overridden	Flag che indica se il codice è stato sovrascritto
m_active	Indica se il job è attivo
m_modified	Flag che indica se il codice è stato modificato
m_deleted	Flag che indica se il codice è stato cancellato
m_config	
m_SVMsource	Codice sorgente

m_sample	Valore
m_LastSampl	Valore precedente letto
m_next	Job successivo al presente job
m_reference;	Generico parametro per referenziare chi possiede il job
upper	Job nella posizione sopra nell'array dei job
lower[MAX_LOWER_COUNT]	Jobs da cui dipende questo job

Valore

Il valore, come accennato è composto da più elementi. Il valore di un Job è una struttura.

TABELLA 12 – MEMBRI PUBBLICI DELLA PRIMITIVA VALUE

MEMBRO	DESCRIZIONE
time	tempo
value	valore
quality	OPC quality
flag	Flag di validità

Il linguaggio

Ora che abbiamo le informazioni relative alla scrittura, sintassi, operatori e ordine di esecuzioni delle operazioni possiamo approfondire i costrutti che compongono il linguaggio.

Prima di tutto approfondiamo il concetto di tempo di un job. Come abbiamo ampiamente analizzato un job appartiene ad un device. Ogni device ha il suo tempo di acquisizione ma anche il job al suo interno ha il suo tempo di acquisizione.

L'ordine di importanza è il seguente:

TEMPO DEL DEVICE > TEMPO DI UN JOB.

Ne consegue che un device con tempo 60 secondi itererà ogni 60 secondi, un job quindi sarà eseguito ogni 60 secondi anche se il suo tempo interno è inferiore. Viceversa, un device con tempo 5 secondi itererà i job al suo interno ogni 5 secondi. Il tempo del job invece serve per far scattare a true il flag *elapsed*.

Cos'è il flag elapsed? Traducendo dall'inglese possiamo renderlo come "scaduto". Un job si dice scaduto quando è trascorso il suo tempo di elapsed. Questa condizione può essere utilizzata a nostro vantaggio per la costruzione di cicli iterativi e per le operazioni I/O come ad esempio il salvataggio in un database.

Osserviamo un esempio di codice:

```
depend "CO";
depend "Portata fumi";
depend "IMPIANTO";

"Flusso di massa CO" #= "CO";
```

```
if ("IMPIANTO" == 34) then
    "Flusso di massa CO" = 0;
    "Flusso di massa CO".flag = 0;
else
    "Flusso di massa CO" = ("CO" * "Portata fumi") / 1000000;
    if (("CO".flag == 0) || ("CO".flag == 20)) then
        if ("Portata fumi".flag == 0) then
            "Flusso di massa CO".flag = 0;
        else
            "Flusso di massa CO".flag = "Portata fumi".flag;
        fi;
    else
        "Flusso di massa CO".flag = "CO".flag;
    fi;
fi;

if elapsed() then
    storep(11);
else
    store(8);
fi;
```

Soltanto le Job di tipo analogico, digitale, uscita analogica e digitale vengono eseguite con il tempo della Device. Le Job di tipo elaborazione hanno bisogno di dipendere da una Job che invece viene eseguita autonomamente.

Ma procediamo per gradi.

Esaminando il codice la prima istruzione che troviamo è la *depend,* la quale svolge quanto detto sopra, fa in modo che il job “Flusso di massa CO” venga eseguito subito dopo i job “CO”, “Portata fumi” e “IMPIANTO”. Questo ci assicura che il valore che assegneremo al job Flusso di massa siano aggiornati al secondo i-esimo di esecuzione.

L’istruzione depend permette l’esecuzione del job ma è comunque necessario assegnare il tempo a tale job. Il tempo viene quindi assegnato con l’operatore #=.

Infine troviamo l’istruzione elapsed. Il seguente job ha un tempo a 5 secondi come la Device, ma supponendo che volessimo archiviare il valore letto ogni 60 secondi, utilizzando l’istruzione elapsed otterremo il seguente risultato.
Per farlo sarà necessario impostare il tempo del job a 60 secondi e poi nel blocco true dell’istruzione “if elapsed” gestiremo l’archiviazione del dato tramite la funzione storep(11).

Le variabili

Le variabili possono essere dichiarate all’interno degli oggetti e nel referenziarle non è necessario utilizzare l’istanza dell’oggetto di riferimento. Le variabili dichiarate non sono fortemente tipizzate e possono essere di tipo INTEGER o REAL. Ogni oggetto istanziato ha il suo ciclo di vita. Le variabili dichiarate sono globali. Un oggetto può ereditare il ciclo di vita di un altro oggetto istanziato.

Ne consegue che ogni variabile dichiarata è condivisa tra tutti i device, di conseguenza possiamo impostare un valore in un device e leggerlo in un altro attraverso una variabile.

Mentre, per accedere alle informazioni di un job in un device diverso da quello di appartenenza dobbiamo referenziare il device di appartenenza tramite il token @.

Ecco un esempio:

```
depend "O2U";
depend "E01_FRAIR@O2U";

"O2U_InJob" := "O2U";

if (bittest("STATI IMPIANTO", 7) == 1) then
     "O2U_InJob" := "E01_FRAIR@O2U";
else
     noop;
fi;

if elapsed() then
  storep(11);
else
  store(8);
fi;
```

Possiamo osservare come al verificarsi di una condizione specifica del digitale impaccato STATI IMPIANTO il valore di O2U venga preso dal device “E01_FRAIR”.

Ulteriori esempi di codice

Di seguito alcuni esempi di come vengono dichiarate le variabili all’interno di una classe e l’utilizzo dei costrutti.

```
#Fondi scala elettrici
declare "CO_ZE_E01" = 0;
declare "CO_SE_E01" = 10000;

#Fondo scala attuale
declare "CO_ZI_E01" = 0;
declare "CO_SI_E01" = 75;

declare "toll_sup_co_E01" = 0;
declare "toll_inf_co_E01" = 0;

if ("CO" > 2^15) then
    "CO" = "CO" - 65535;
else
    noop;
fi;

#Linearizzazione
"CO" = ("CO_SI_E01" - "CO_ZI_E01") / ("CO_SE_E01" - "CO_ZE_E01") * "CO" - ("CO_SI_E01" -
"CO_ZI_E01") / ("CO_SE_E01" - "CO_ZE_E01") * "CO_ZE_E01";
```

```
"CO".flag = 0;

#SOGLIA SUPERIORE
#-------------------------------------

#Se il CO è superiore al fondo scala + tolleranza 5% VALIDO OVR
if("CO" > ("CO_SI_E01" + "toll_sup_co_E01")) then
  "CO".flag = 21;
   setquality(192);
  "CO" = "CO_SI_E01" + "toll_sup_co_E01";
else
   noop;
fi;
```

Ecco un altro esempio di codice che indica come dichiarare una variabile e utilizzare il valore di un'altra primitiva dichiarata nel codice.

```
depend "O2S";
depend "O2U";

"H2O" := "O2S";

if (("O2U".flag != 0) && ("O2U".flag != 20) && ("O2U".flag != 21)) then
   "H2O".flag = 15;
else
   noop;
fi;

"H2O" = (1 - ( "O2U" / "O2S")) * 100;


if ("H2O" < 0) then
   "H2O" = 0;
else
    noop;
fi;

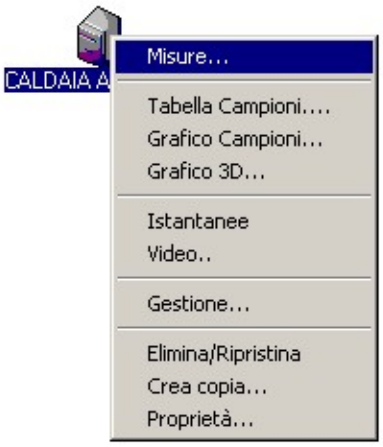
if elapsed() then
   storep(11);
else
   store(8);
fi;
```

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

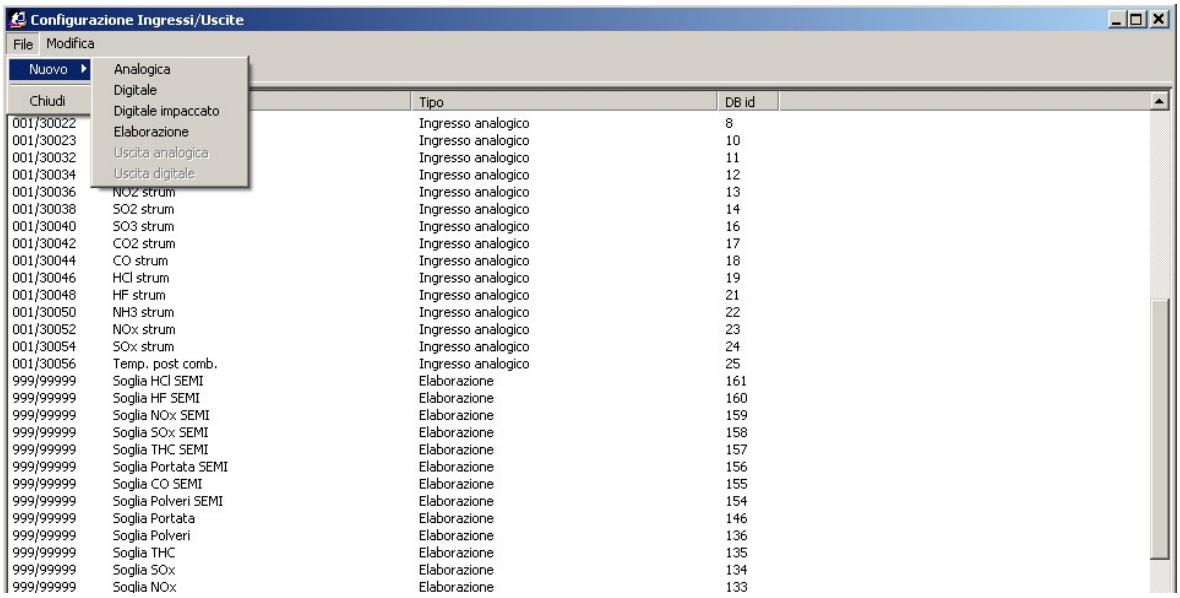
P.IVA 02693200426

Di seguito un estratto di manuale che illustra una procedura operativa per scrivere in linguaggio Manager.

Una volta entrati in modalità di configurazione, per creare/modificare un nuovo Job occorre fare click con il tasto destro del mouse in corrispondenza della Device cui va riferita e selezionare la voce *Job…* dal menu a tendina che appare:



Dalla maschera che appare, è possibile creare un nuovo Job (selezionando la voce *Nuovo* del menu *File* e scegliendo la tipologia di interesse) oppure modificare un Job esistente (facendo doppio click con il mouse sulla Job di interesse).

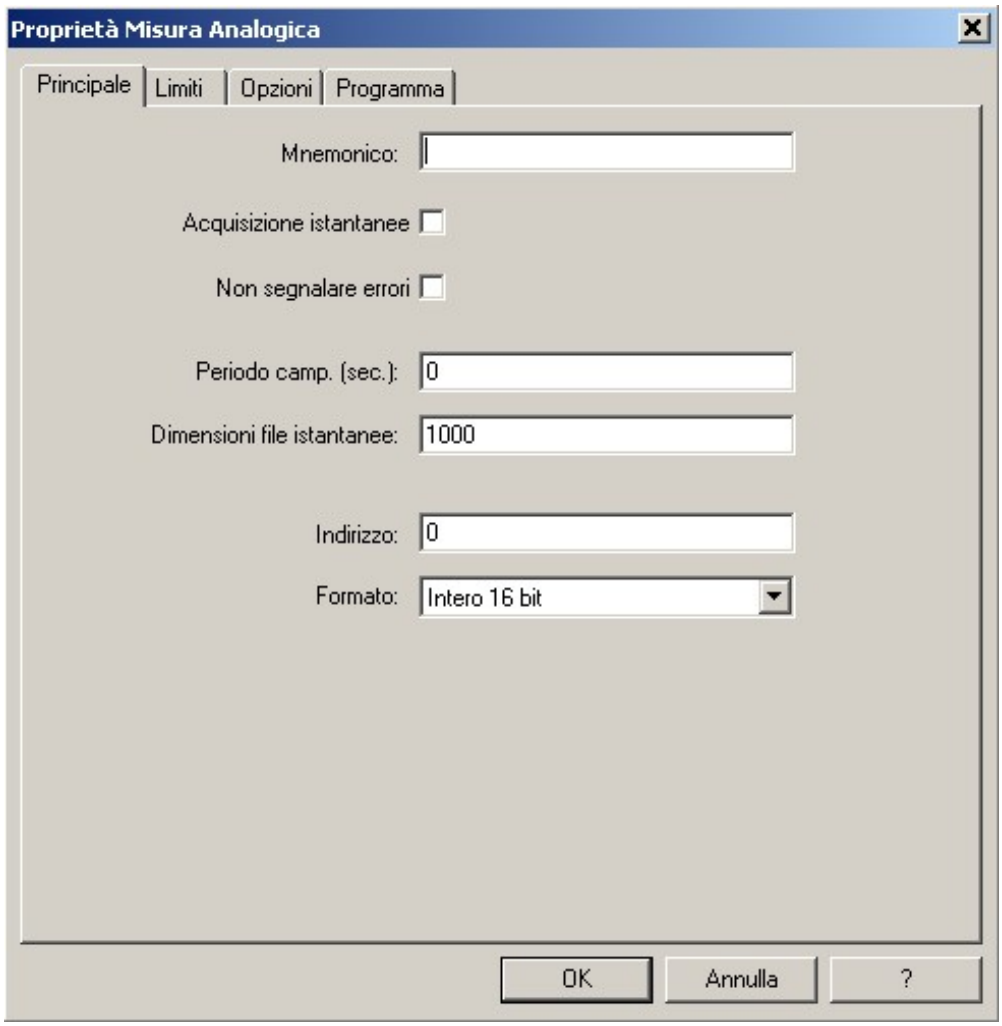


		Tipo	DB id
001/30022		Ingresso analogico	8
001/30023		Ingresso analogico	10
001/30032		Ingresso analogico	11
001/30034		Ingresso analogico	12
001/30036	NO2 strum	Ingresso analogico	13
001/30038	SO2 strum	Ingresso analogico	14
001/30040	SO3 strum	Ingresso analogico	16
001/30042	CO2 strum	Ingresso analogico	17
001/30044	CO strum	Ingresso analogico	18
001/30046	HCl strum	Ingresso analogico	19
001/30048	HF strum	Ingresso analogico	21
001/30050	NH3 strum	Ingresso analogico	22
001/30052	NOx strum	Ingresso analogico	23
001/30054	SOx strum	Ingresso analogico	24
001/30056	Temp. post comb.	Ingresso analogico	25
999/99999	Soglia HCl SEMI	Elaborazione	161
999/99999	Soglia HF SEMI	Elaborazione	160
999/99999	Soglia NOx SEMI	Elaborazione	159
999/99999	Soglia SOx SEMI	Elaborazione	158
999/99999	Soglia THC SEMI	Elaborazione	157
999/99999	Soglia Portata SEMI	Elaborazione	156
999/99999	Soglia CO SEMI	Elaborazione	155
999/99999	Soglia Polveri SEMI	Elaborazione	154
999/99999	Soglia Portata	Elaborazione	146
999/99999	Soglia Polveri	Elaborazione	136
999/99999	Soglia THC	Elaborazione	135
999/99999	Soglia SOx	Elaborazione	134
999/99999	Soglia NOx	Elaborazione	133

Nelle pagine seguenti sono descritti i parametri di configurazione di ciascuna tipologia di Job relativa a ciascuna tipologia di Device.

Job Analogica

La finestra di configurazione della Job di tipo analogico su Device SIEMENS PLC S5/S7 è caratterizzata da n.4 sezioni: *Principale*, *Limiti*, *Opzioni* e *Programma*.



Nella sezione *Principale* occorre compilare i seguenti campi:

Mnemonico: Stringa rappresentante il nome da attribuire alla Job

Acquisizione istantanee: Flag per abilitare l'acquisizione dal campo

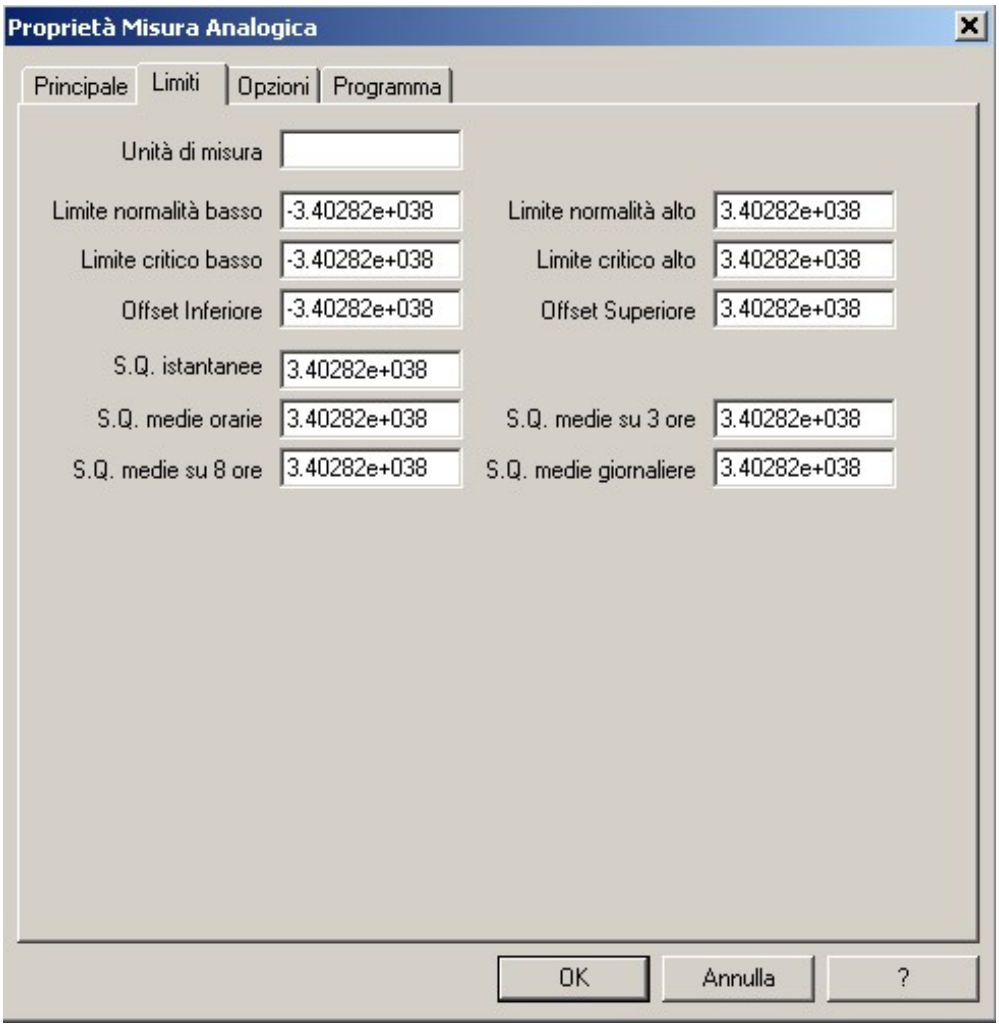
Non segnalare errori: Flag per disabilitare la segnalazione di anomalie nell'acquisizione

Periodo camp. (sec.): Parametro cui riferirsi all'interno del programma (vedi sezione *Programma*), tipicamente per la frequenza di archiviazione su database

Dimensioni file istantanee: Parametro che indica le dimensioni in KB del file in cui sono archiviati, in formato binario, i valori istantanei

Indirizzo: Parametro che indica l'indirizzo del PLC cui puntare in lettura

Formato: Dal menu a tendina occorre selezionare il formato in cui sono rappresentati i valori nell'area di lettura (*Intero 16 bit* oppure *Virgola mobile 32 bit*)



Nella sezione *Limiti* occorre compilare i seguenti campi:

Unità di Job: Stringa corrispondente all'unità di Job del parametro in acquisizione

Limite normalità basso: Soglia inferiore di normalità (valori inferiori sono considerati critici)

Limite critico basso: Soglia inferiore di criticità (valori inferiori sono considerati in allarme)

Offset Inferiore: Soglia inferiore validità (valori inferiori sono considerati non validi)

Limite normalità alto: Soglia superiore di normalità (valori superiori sono considerati critici)

Limite critico alto: Soglia superiore di criticità (valori superiori sono considerati in allarme)

Offset Superiore: Soglia superiore di validità (valori superiori sono considerati non validi)

S.Q. istantanee: Non usato

S.Q. medie orarie: Non usato

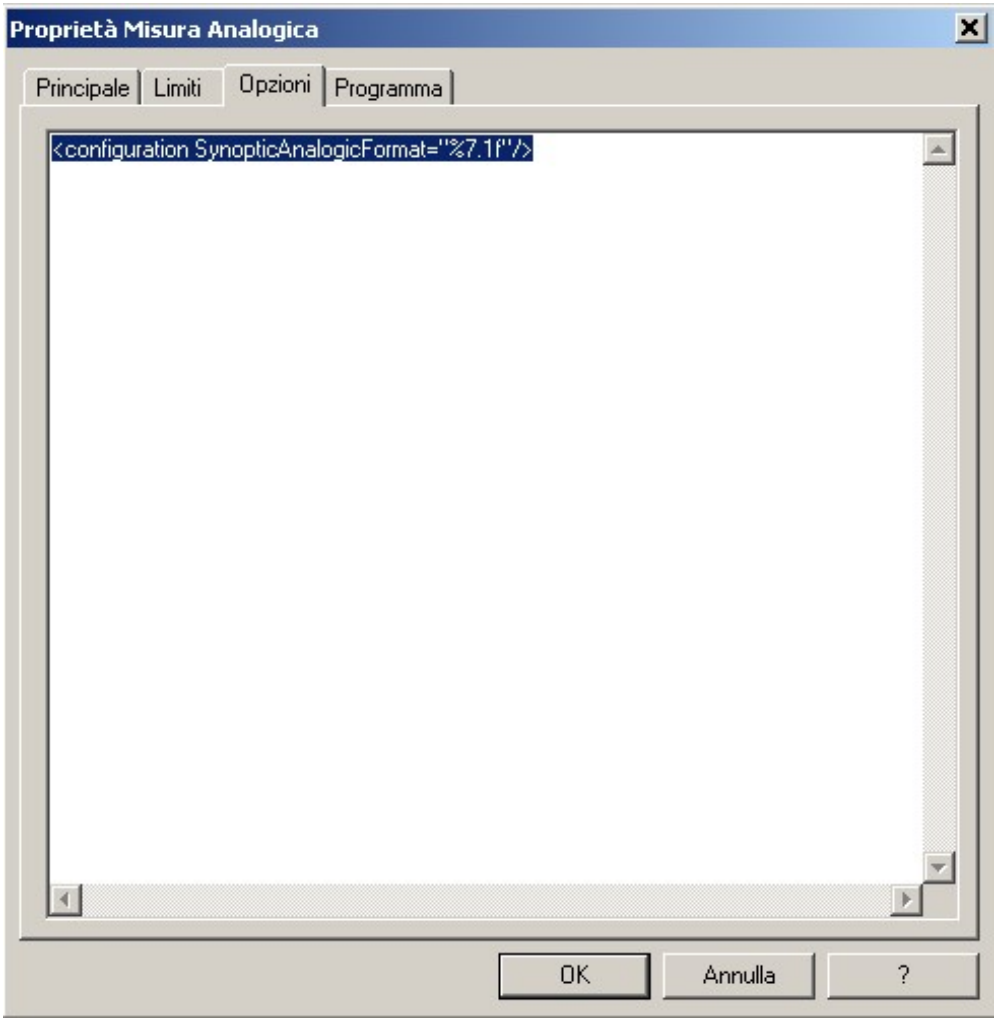
S.Q. medie su 3 ore: Non usato

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

S.Q. medie su 8 ore: Non usato

S.Q. giornaliere: Non usato



Nella sezione *Opzioni* va scritto, in linguaggio proprietario, il codice che si intende eseguire *run time* ad ogni acquisizione.

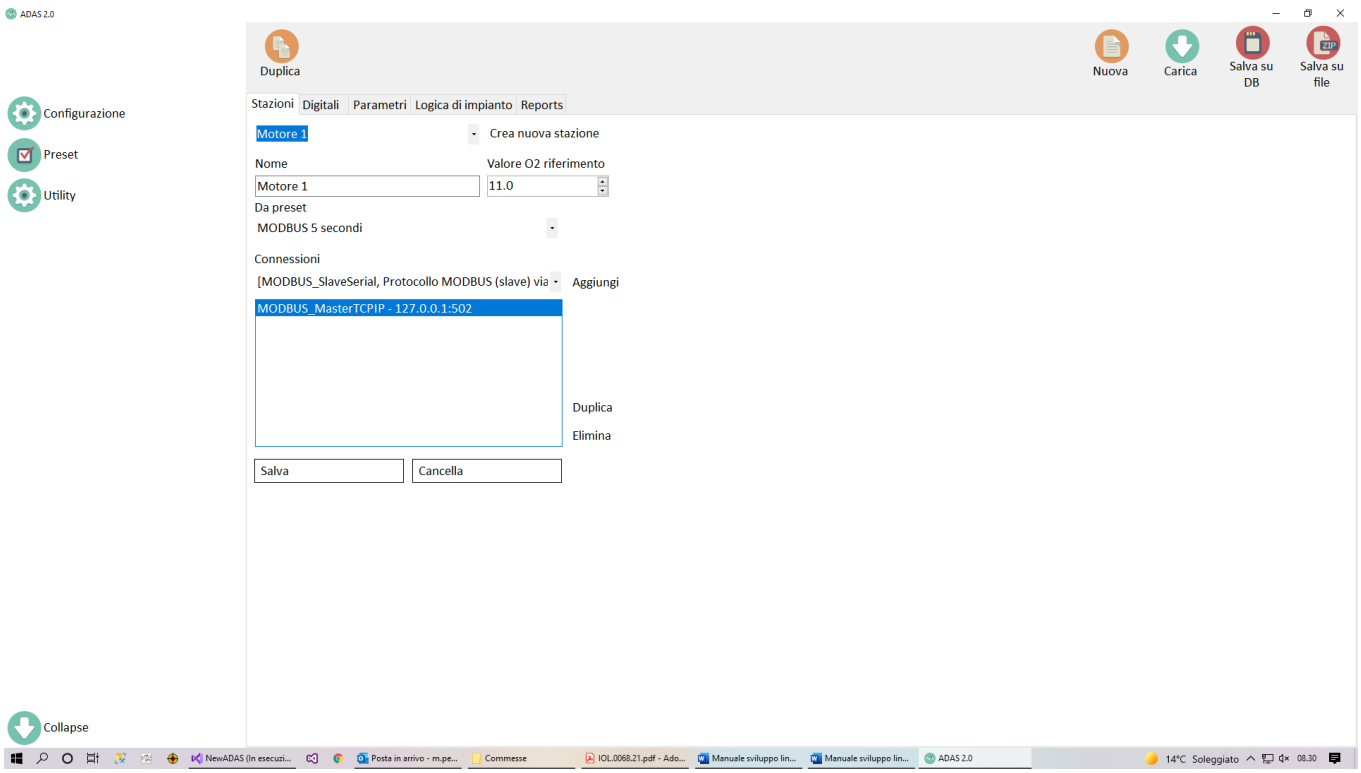
Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

Editor visuale di codice

Attraverso l'applicativo ADAS 2.0 è possibile scrivere codice in linguaggio Manager visualmente.

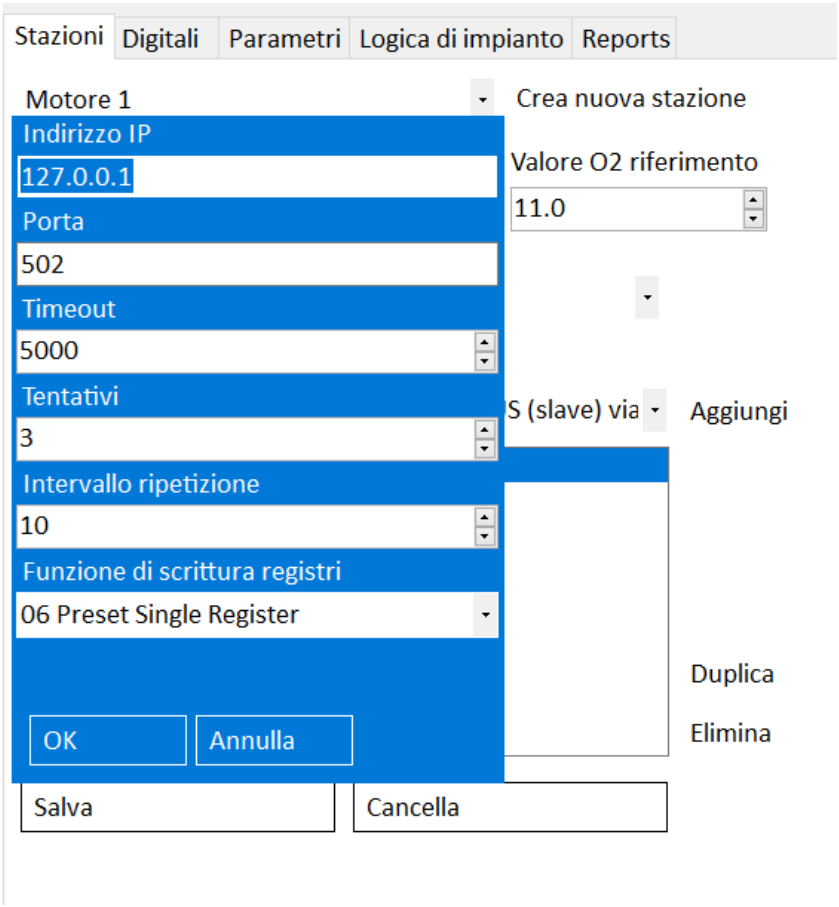
L'applicativo permette la creazione dei device e dei job a essi associati e contiene dei preset che possono essere applicati in maniera scalare.



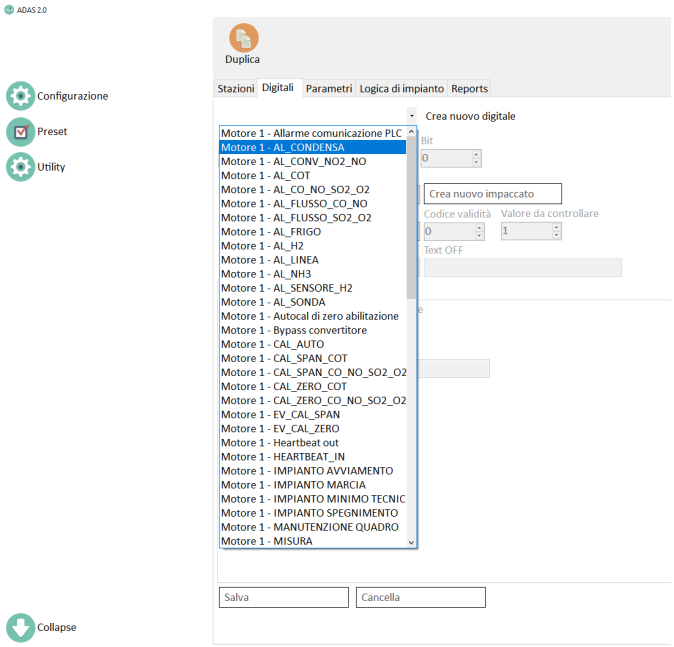
La creazione di un device ci consente poi di associare ad esso i job che andremo a creare. Per ogni device è anche possibile specificare il protocollo di acquisizione e l'indirizzo o porta seriale da cui acquisire.

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426



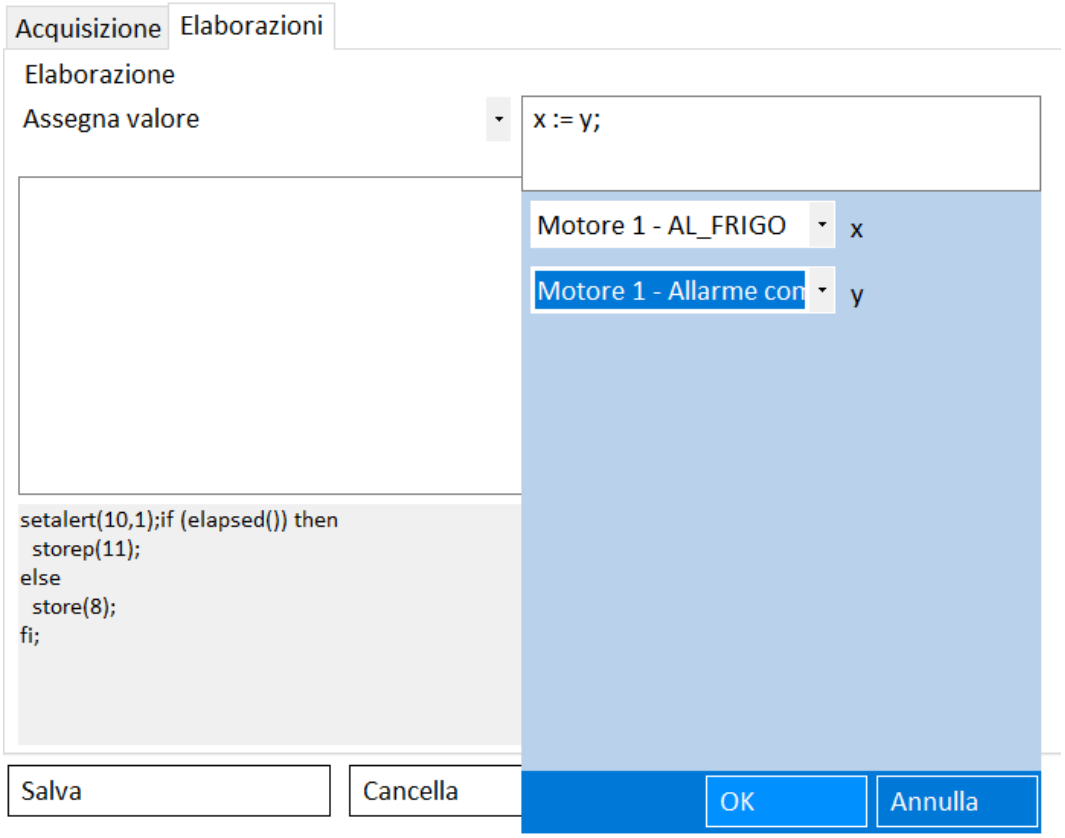
Una volta stabilita la connessione possiamo procedere creando Job digitali e analogiche. Il codice verrà generato in automatico utilizzando le impostazioni che andremo a stabilire nel software.



Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / **www.menerva.it**

P.IVA 02693200426

Nel creare i job è possibile accedere ad una libreria di elaborazioni già esistenti e parametrizzate. Tali elaborazioni possono poi essere aggiunte e tramite una finestra di controllo potremo scegliere quale job associare a quale parametro.

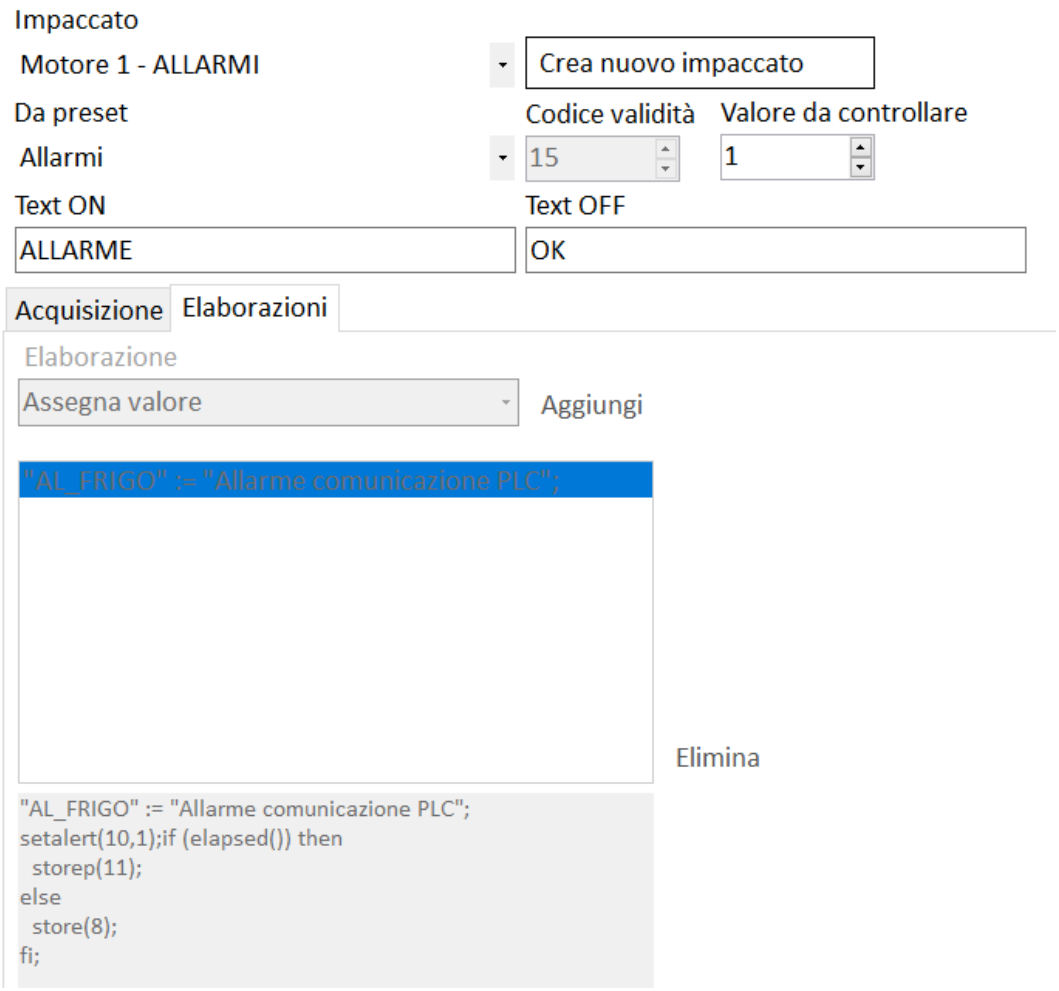


Come si vede nell'immagine possiamo riconoscere un token del linguaggio Manager, il due punti uguale (:=).

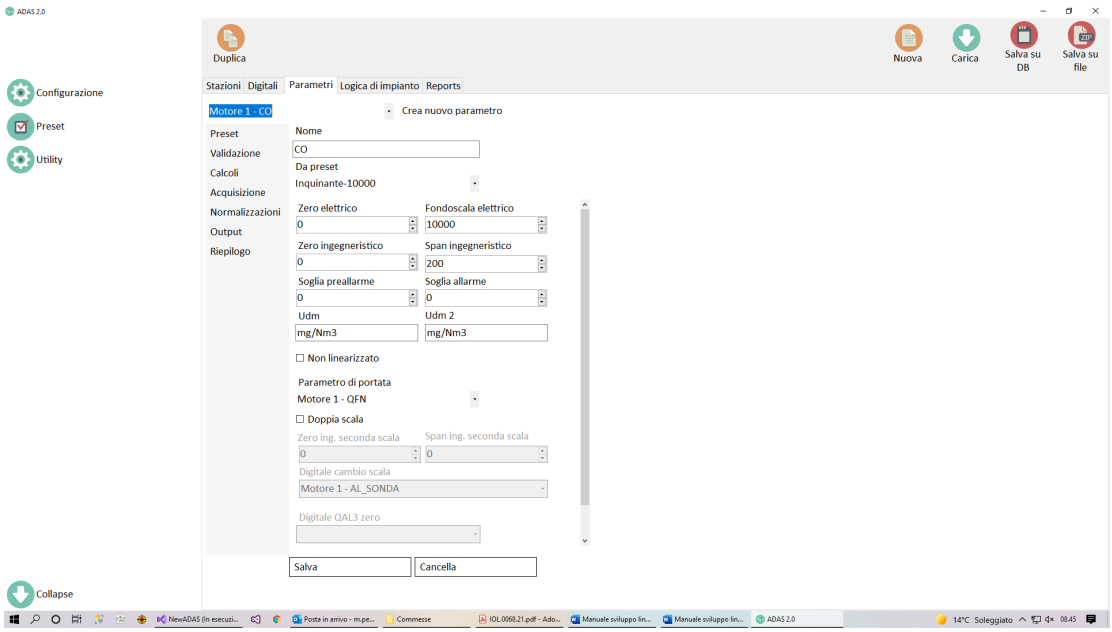
Questo token che come abbiamo visto assegna tempo, valore e flag di un job ad un altro in questa elaborazione è parametrizzato, x prende il tempo, valore e flag di y. Quindi il software ci chiede di associare al segnaposto x un job e al segnaposto y un altro job.

Confermando genererà in automatico il codice dell'assegnazione.

Come possiamo inoltre notare del codice è già presente, troviamo infatti l'istruzione setalert e l'archiviazione del dato come abbiamo già analizzato nei capitoli precedenti. Queste due istruzioni sono state generate in automatico dalle impostazioni che abbiamo dato nel creare il digitale, ovvero selezionando genera allarme e abilitando l'archiviazione dei valori letti.



Dopo aver configurato i job di tipo digitale è consigliato configurare i job di tipo analogico. La sezione relativa ai job analogici è la tab Parametri.



Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / **www.menerva.it**

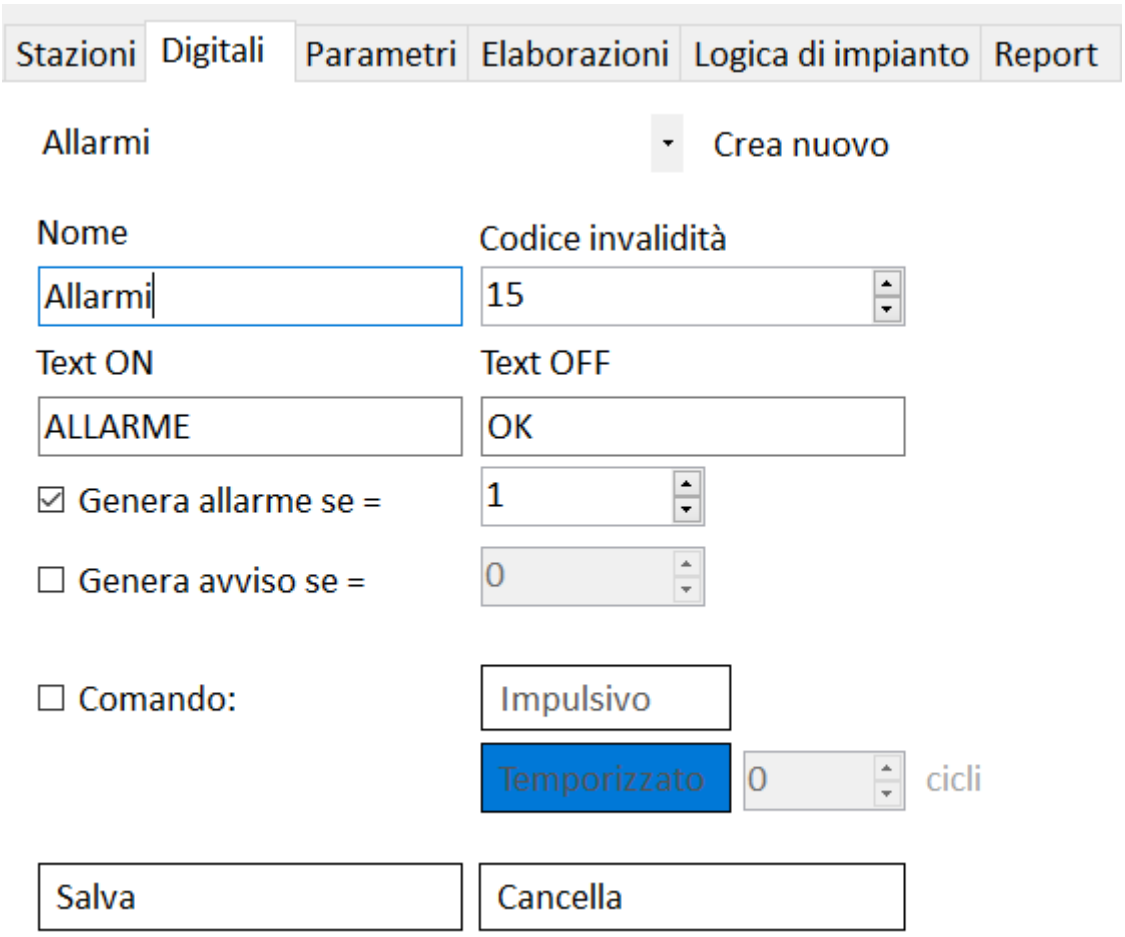
P.IVA 02693200426

I valori che andremo ad impostare li ritroveremo convertiti in codice.

Come avviene questo?

Introduciamo il concetto di preset. Un Preset è un insieme di settaggi che compongono un job. Determina il codice che conterrà e i job aggiuntivi che dovranno essere generati per svolgere quanto necessario.

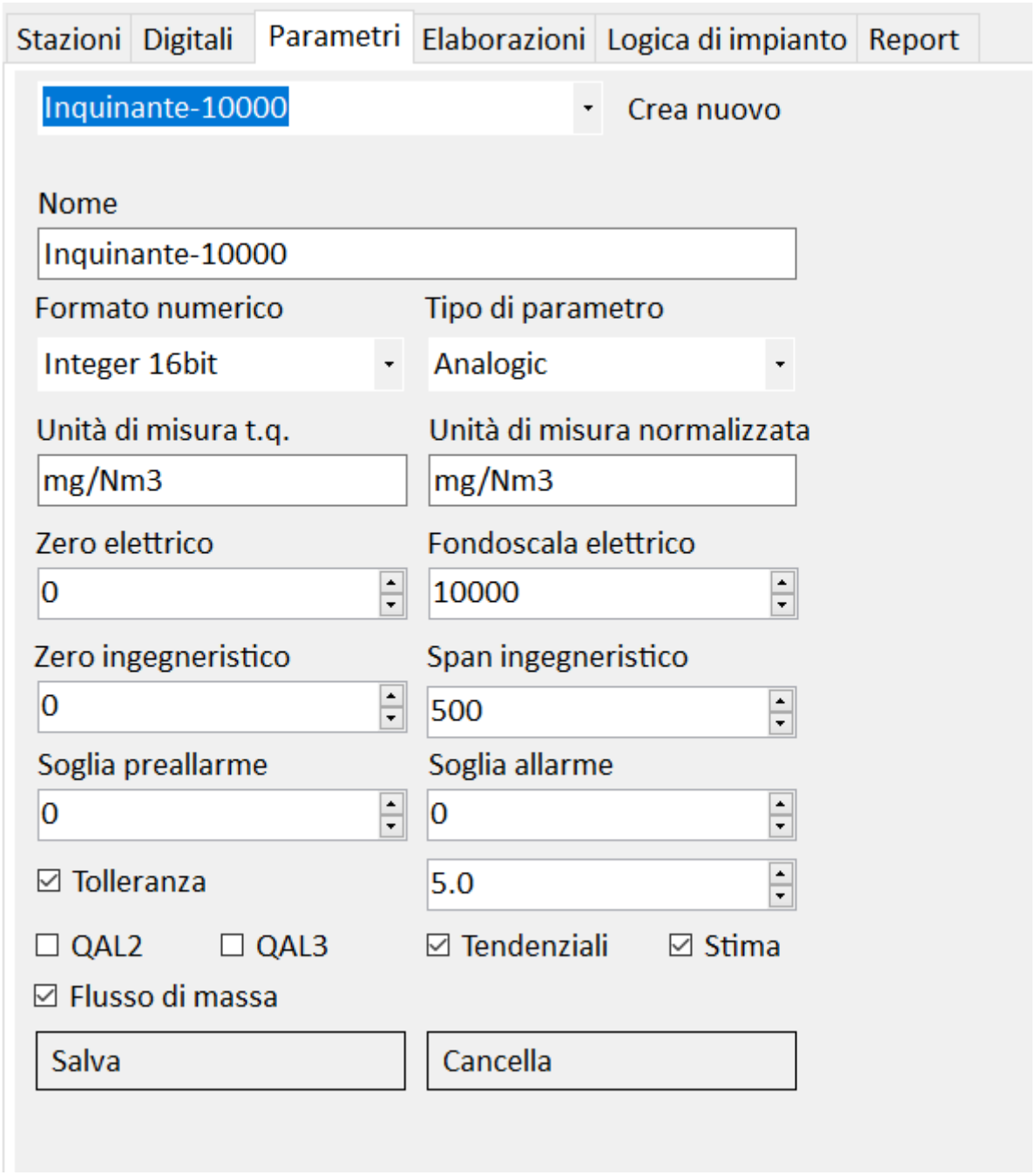
Osserviamo la finestra di configurazione di un preset digitale.



Abbiamo il nome del preset, il codice che assegna al flag delle Job che lo implementano, il testo da mostrare a video quando è acceso e spento e le istruzioni se generare un allarme, un avviso oppure se si tratta di un comando.

Come abbiamo visto precedentemente il setalert che abbiamo visto nell'esempio di poco fa viene generato automaticamente perché il digitale in questione ha attivo il settaggio "genera allarme".

Osserviamo ora la configurazione di un preset analogico.



I valori impostati verranno convertiti in codice. Ad esempio, i valori di fondoscala e span saranno trasformati in variabili dichiarate e utilizzati per la linearizzazione del segnale acquisito, ovvero la conversione di un segnale elettrico che va da un range di 0 a 10000 ad un valore ingegneristico che va da 0 a 500, la cui unità di Job come leggiamo in “Unità di Job” è mg/Nm^3.

Cosa significa questo?

Recuperiamo un frammento di codice che abbiamo già analizzato in precedenza.

```
#Fondi scala elettrici
```

```
declare "CO_ZE_E01" = 0;
declare "CO_SE_E01" = 10000;

#Fondo scala attuale
declare "CO_ZI_E01" = 0;
declare "CO_SI_E01" = 500;

declare "toll_sup_co_E01" = 1.05;
declare "toll_inf_co_E01" = 0.05;

if ("CO" > 2^15) then
    "CO" = "CO" - 65535;
else
    noop;
fi;

#Linearizzazione
"CO" = ("CO_SI_E01" - "CO_ZI_E01") / ("CO_SE_E01" - "CO_ZE_E01") * "CO" - ("CO_SI_E01"
- "CO_ZI_E01") / ("CO_SE_E01" - "CO_ZE_E01") * "CO_ZE_E01";

"CO".flag = 0;

#SOGLIA SUPERIORE
#-----------------------------------

#Se il CO è superiore al fondo scala + tolleranza 5% VALIDO OVR
if("CO" > ("CO_SI_E01" * "toll_sup_co_E01")) then
  "CO".flag = 21;
   setquality(192);
  "CO" = "CO_SI_E01" * "toll_sup_co_E01";
else
   noop;
fi;
```

Come possiamo vedere, il fondoscala elettrico corrisponde alla variabile “CO_SE_E01” e lo span ingegneristico alla variabile “CO_SI_E01”.

La linearizzazione all’algoritmo:

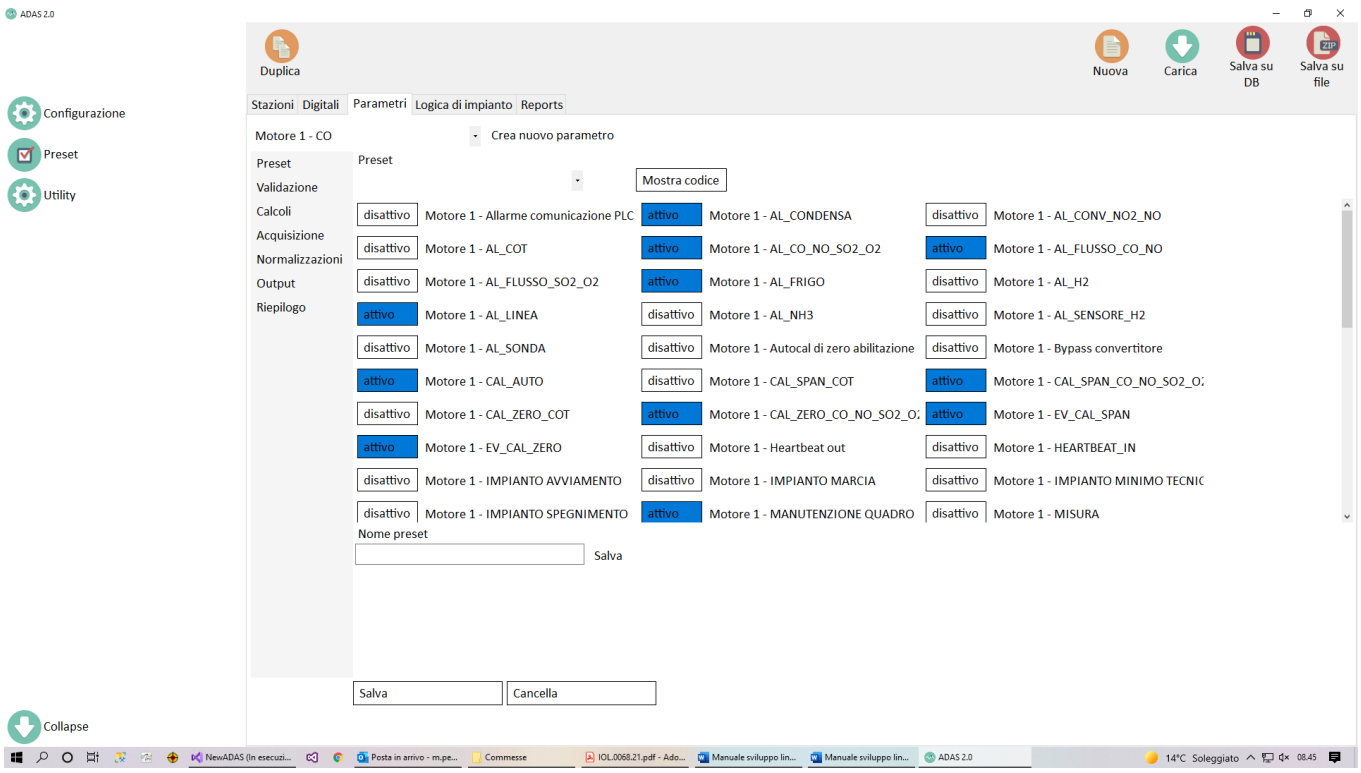
```
"CO" = ("CO_SI_E01" - "CO_ZI_E01") / ("CO_SE_E01" - "CO_ZE_E01") * "CO" - ("CO_SI_E01"
- "CO_ZI_E01") / ("CO_SE_E01" - "CO_ZE_E01") * "CO_ZE_E01";
```

La tolleranza corrisponde alle due variabili “toll_sup_co_E01” e “toll_sup_co_E01”

Gli altri check che vediamo, ovvero QAL2, QAL3, Tendenziali, Stima e Flusso di massa determinano la creazione di altri job collegati al job padre e che implementeranno le logiche relative alla QAL2, QAL3 etc.. .

È possibile dopodiché implementare la validazione del job, ovvero stabilire quali segnali digitali rendono valida o meno la misurazione effettuata.

Per far questo basta scegliere i digitali precedentemente creati e che, come abbiamo visto implementano i preset con i relativi codici di validità o invalidità.



Dopo averli selezionati il software genera in automatico il codice in linguaggio Manager.

Eccone un estratto:

```
declare "CO_TollSup_Motore 1" = 1.05;
declare "CO_TollInf_Motore 1" = 0.05;
declare "CO_ZE_Motore 1" = 0;
declare "CO_SE_Motore 1" = 10000;
declare "CO_ZI_Motore 1" = 0;
declare "CO_SI_Motore 1" = 200;

if ("CO" > 2^15) then
    "CO" = "CO" - 65535;
else
    noop;
fi;

"CO" = ((("CO_SI_Motore 1" - "CO_ZI_Motore 1") / ("CO_SE_Motore 1" - "CO_ZE_Motore
1"))) * "CO" + ("CO_ZI_Motore 1" - ((("CO_SI_Motore 1" - "CO_ZI_Motore 1") /
("CO_SE_Motore 1" - "CO_ZE_Motore 1")) * "CO_ZE_Motore 1"));

"CO".flag = 0;

if ("CO" > ("CO_SI_Motore 1" * "CO_TollSup_Motore 1")) then
"CO".flag = 21;
"CO" = "CO_SI_Motore 1" * "CO_TollSup_Motore 1";
else
noop;
fi;

if (("CO" < "CO_ZI_Motore 1") &&  ( "CO" > ("CO_SI_Motore 1" * "CO_TollInf_Motore 1" *
-1))) then
"CO" = "CO_ZI_Motore 1";
```

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / **www.menerva.it**

P.IVA 02693200426

```
else
noop;
fi;
if ("CO" < ("CO_SI_Motore 1" * "CO_TollInf_Motore 1" * -1)) then
"CO".flag = 151;
else
noop;
fi;

if (bittest("ALLARMI", 11)) then
"CO".flag = 15;
else
noop;
fi;

if (bittest("ALLARMI", 12)) then
"CO".flag = 15;
else
noop;
fi;

if (bittest("ALLARMI", 0)) then
"CO".flag = 15;
else
noop;
fi;

if (bittest("ALLARMI", 10)) then
"CO".flag = 15;
else
noop;
fi;

if (bittest("ALLARMI", 9)) then
"CO".flag = 15;
else
noop;
fi;

if (bittest("STATI SISTEMA", 9)) then
"CO".flag = 40;
else
noop;
fi;

if (bittest("STATI SISTEMA", 11)) then
"CO".flag = 42;
else
noop;
fi;

if (bittest("STATI SISTEMA", 10)) then
"CO".flag = 41;
else
noop;
fi;

if (bittest("STATI IMPIANTO", 9)) then
"CO".flag = 10;
else
noop;
fi;

if (bittest("STATI IMPIANTO", 8)) then
"CO".flag = 10;
else
noop;
```

Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / www.menerva.it

P.IVA 02693200426

menerva

```
fi;

if (bittest("STATI IMPIANTO", 5)) then
"CO".flag = 10;
else
noop;
fi;
```

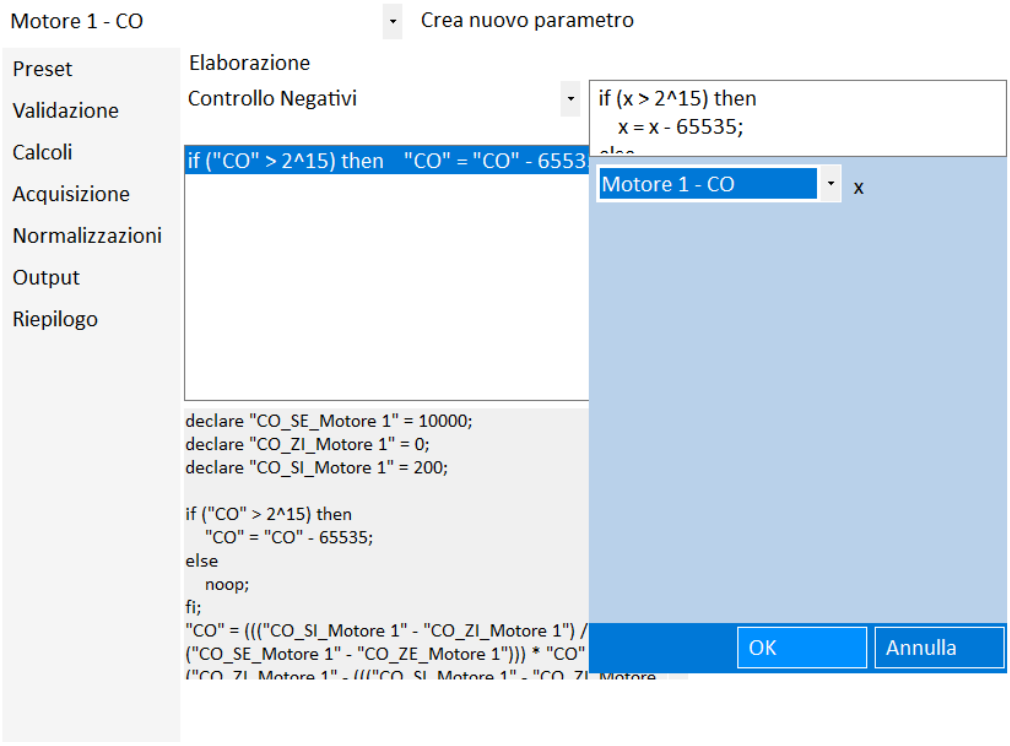
```
if (bittest("Esteem_CO_Enabled", 0)) then
"CO" = "CO_Estimated";
setquality(192);
else
noop;
fi;

if (elapsed()) then
  storep(11);
else
  store(8);
fi;
```

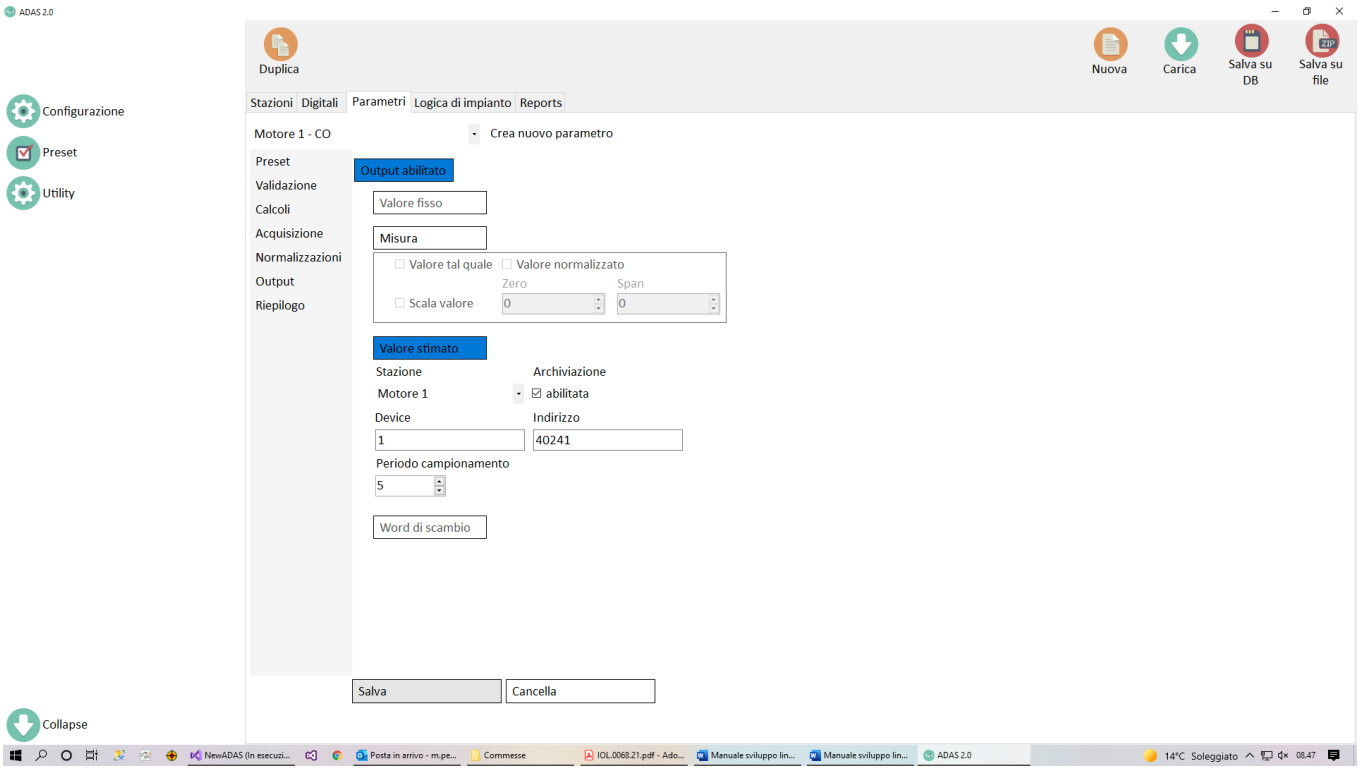
La parte evidenziata corrisponde all'implementazione della validazione.

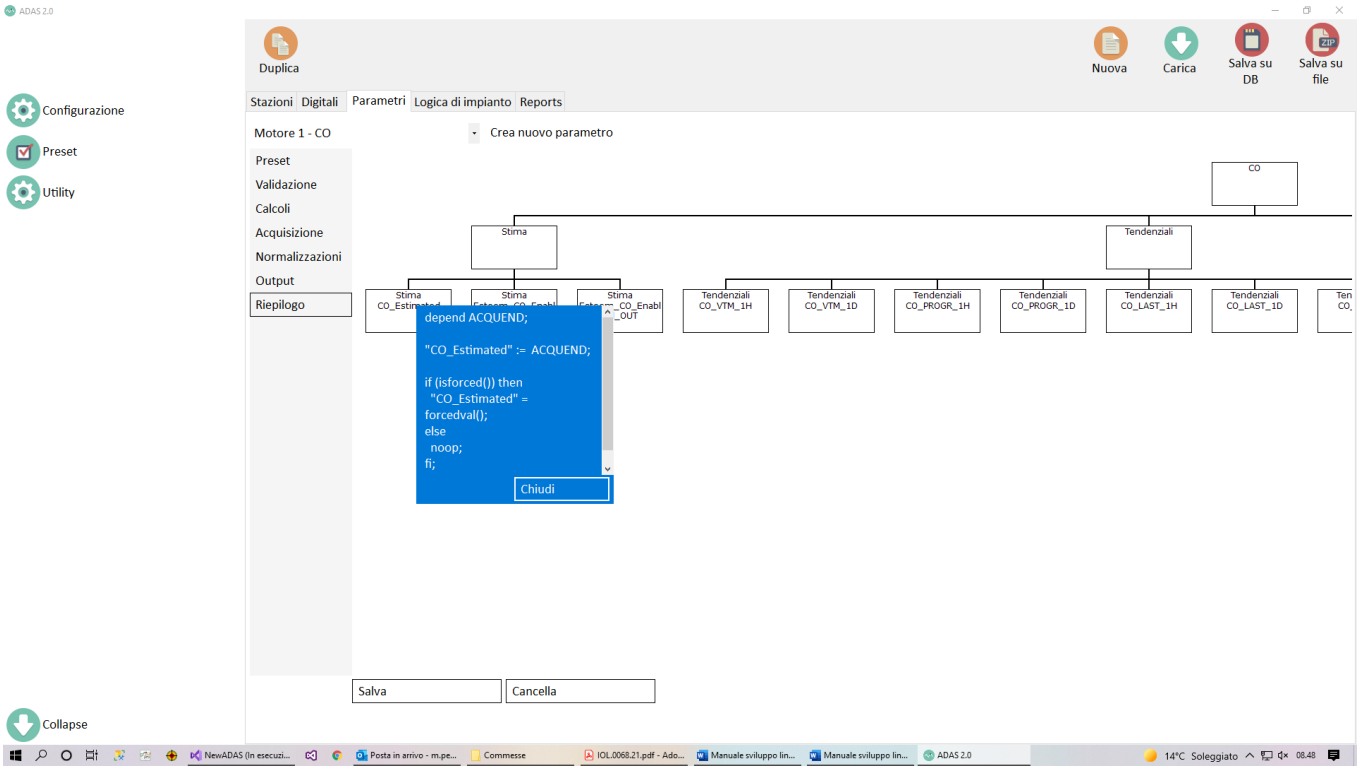
La funzione bittest testa il valore del digitale impaccato, il numero che segue corrisponde al bit da testare e viene assegnato il flag che abbiamo configurato precedentemente nella fase di creazione dei digitali.

Anche per i parametri è possibile implementare le elaborazioni preconfigurate.



Inoltre, è possibile configurare le uscite e consultare il grafico di tutti i job che verranno creati e visualizzare il codice che implementano.





Menerva s.r.l.s.
Chiaravalle c.so G. Matteotti, 91 60033 (AN)
Tel/Fax 071 7457050
info@menerva.it / **www.menerva.it**

P.IVA 02693200426

Riferimenti

[1] https://www.gnu.org/gnu/gnu-history.it.html

[2] https://www.gnu.org/software/bison/manual/html_node/Introduction.html

[3] https://standards.iso.org/ittf/PubliclyAvailableStandards/s026153_ISO_IEC_14977_1996(E).zip

[4] https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap09.html